

Got it — they didn't render on your end. Let me resend all three one by one right now.

PROJECT REQUIREMENT DOCUMENT

Project 1 — Spatiotemporal Graph Engine

Flagship Research Contribution | ML/Data Backend

1. Project Identity

Field	Detail
Full title	Temporal Graph Alignment and Multi-Agent Role Permutation for Explainable Football Tactical Analysis
Sub-project title	Spatiotemporal Graph Engine — Computational Backend
Project type	Research system + production-grade ML backend
Role on resume	Machine Learning Engineer / Data Engineer / Backend Scientist
Publication target	IEEE Transactions on Games / IEEE ICASSP / ACM SIGKDD
Priority	Flagship — must be built first, others depend on this

2. Problem This Project Solves

Current football analytics tools provide either static summary statistics or isolated event visualizers. They fail to capture how team structures and player roles evolve dynamically across match phases. Existing methods either require expensive continuous optical tracking data or reduce tactical states to rigid pre-defined formation templates, ignoring the fluid nature of elite-level play.

This project builds the mathematical and computational core that solves this — a system that learns time-varying representations of team structures, detects tactical phase transitions, and decouples player identities from active tactical roles using principled mathematical formulations.

3. Research Contributions (Novelty)

Contribution 1 — Bipartite Role-Permutation Modeling

Players continuously swap positions on the pitch. Assuming a player's role is fixed by their starting position introduces noise into xT/VAEP calculations. This project introduces a dynamic bipartite matching layer using the Hungarian algorithm that solves the assignment problem frame-by-frame, decoupling player identities from active tactical roles.

Contribution 2 — Physically Constrained State Parsing

Tactical phase transitions are modeled as a sequential edge-selection problem on a fully connected temporal graph. A Dynamic Masked CRF with Viterbi decoding enforces domain-specific physical constraints — preventing the model from predicting impossible transitions like an instant shift from deep defensive block to high press.

Contribution 3 — Topological Semantic Grounding

LLMs fail at raw coordinate reasoning. This project maps high-dimensional geometric configurations (Delaunay triangulations, convex hulls, team centroids) into a structured semantic attribute space before any language model sees them. This eliminates coordinate hallucination and gives Project 3 a reliable structured input.

4. Research Questions

RQ	Question	Hypothesis	Metric
RQ 1	Can TVG representations of passing networks detect tactical transitions without continuous tracking?	TVG with temporal edge-weight decay identifies macro-tactical shifts within a 3-minute window matching tracking-based accuracy	Temporal F1-Score vs manual annotations
RQ 2	Does bipartite role matching via a hidden permutation matrix improve possession-based threat valuation?	Disentangling player identities from tactical roles reduces noise in xT/VAEP yielding statistically significant ROC-AUC improvement	ROC-AUC for possession outcomes

RQ 3	Can topological semantic grounding reduce spatial hallucinations in the agentic layer?	Structured geometric-to-semantic mapping reduces hallucinations by 90%+ compared to single-agent RAG	MRR + human expert factuality evaluation
------	--	--	--

5. System Architecture

Raw Data Layer

- StatsBomb Event JSON → statsbombpy parser
- StatsBomb 360 Context → spatial event frames
- SkillCorner Tracking CSV → 25Hz broadcast coordinates
- Wyscout Event Stream → generalizability validation

Ingestion and Normalization

- kloppy → deserialize and normalize tracking
- socceraction → convert events to SPADL schema
- databallpy → synchronize event and tracking timelines
- Downsample to 5Hz → Redis cache for pre-computed topologies

Feature Engineering Layer

- SPADL actions → xT calculation
- VAEP model → action valuation
- Passing network builder → NetworkX directed weighted graphs
- Convex hull computation → shapely
- Delaunay triangulation → shapely + scipy
- Team centroid tracking → numpy spatial mean
- Voronoi spatial control → scipy.spatial.Voronoi

Graph Representation Layer

- Time-Varying Graph $G(t)$ → PyTorch Geometric temporal graphs
- Node features → player role, position, centrality
- Edge features → pass frequency, xT value, temporal decay
- Spatiotemporal GNN → PyTorch Geometric GCN/GAT backbone

Role Permutation Layer

- Hungarian algorithm → scipy.optimize.linear_sum_assignment
- Cost matrix → Wasserstein distance to formation template
- Dynamic role assignments → frame-by-frame player to role mapping

State Parsing Layer

- Dynamic Masked CRF → torchcrf
- Viterbi decoding → illegal transition masking
- Tactical phase labels → buildup / progression / attack / transition / defensive

Semantic Grounding Layer

- Role-aggregated attributes → per-role geometric descriptors
- Semantic attribute tree → structured JSON for agent consumption
- Topology export → served via FastAPI to Project 3

Storage Layer

- PostgreSQL on Supabase → matches, events, computed metrics, role assignments
- Redis on Upstash → cached spatial topologies at 5Hz
- MLflow → experiment tracking and ablation results

API Layer

- FastAPI → async REST endpoints for Projects 2 and 3
-

6. Dataset Plan

Dataset	Modality	Volume	Role	Access
StatsBomb Open Data	Event stream	1000+ matches	Primary training for xT, VAEP, passing networks	github.com/statsbomb/open-data
StatsBomb 360	Spatial event context	Select tournaments	Localized graph construction	Same repo, select competitions
SkillCorner Open Data	Broadcast tracking 25Hz	10 matches	Off-ball runs, formation tracking, ablation study 2	github.com/SkillCorner/openda
Wyscout Open Dataset	Multi-league events	Full 2017/18 season	Generalizability validation	figshare / Pappalardo et al.

Metrica Sports	Synchronized tracking + events	3 games	Role permutation module validation only	github.com/metrica-sports/sample-data
----------------	--------------------------------	---------	---	---

7. Modules and Build Order

Module	Libraries	V1 or Later
Data ingestion pipeline	statsbombpy, kloppy, socceraction, databallpy	V1
SPADL converter	socceraction	V1
xT calculator	socceraction	V1
VAEP model	socceraction, sklearn	V1
Passing network builder	NetworkX	V1
Centrality engine	NetworkX	V1
Spatial geometry engine	shapely, scipy	V1
Time-varying graph builder	PyTorch Geometric	V1
Hungarian role matcher	scipy.optimize	V1
Semantic grounding export	Custom + FastAPI	V1
PostgreSQL schema	SQLAlchemy + Supabase	V1
Redis cache	Redis + Upstash	V1
FastAPI server	FastAPI, Uvicorn	V1
MLflow tracking	MLflow	V1
CRF state parser	torchcrf	V2
Spatiotemporal GNN	PyTorch Geometric	V2
Celery workers	Celery + Redis	V2

8. Evaluation Plan

Module	Baseline	Metric
Formation classifier	Static lineup templates + Rolling Wasserstein + HDS-SGT	Classification accuracy + transition F1
Role permutation	Static coordinate mappings + simple spatial clustering	Permutation error rate + transport cost
Valuation model	Action-only VAEP + xT Markov baseline	ROC-AUC
Ablation 1	With vs without Hungarian matching	Formation accuracy delta
Ablation 2	Events only vs events + SkillCorner off-ball	Valuation ROC-AUC delta
Ablation 3	CRF on vs CRF off	Transition detection F1 delta

9. Tech Stack

Layer	Choice
Language	Python 3.11+
Data parsing	statsbombpy, kloppy, socceraction, databallpy
Graph and ML	PyTorch Geometric, NetworkX, scipy
Geometry	shapely, scipy.spatial
Valuation	socceraction (xT + VAEP)
Storage	PostgreSQL on Supabase
Cache	Redis on Upstash
API	FastAPI + Uvicorn
Task queue	Celery + Redis
Experiment tracking	MLflow
Containerization	Docker + Docker Compose

Deployment	Google Cloud Run
CI/CD	GitHub Actions

10. Diagrams to Create

1. System context diagram
 2. Layer and container diagram
 3. Data flow diagram — raw JSON to SPADL to graph to semantic JSON
 4. Component diagram — module relationships
 5. ERD — matches, events, players, roles, graphs, metrics
 6. Deployment diagram — Cloud Run, Supabase, Upstash
-

11. Deliverables

- Fully functional data ingestion and normalization pipeline
- xT and VAEP computed for all StatsBomb matches
- Passing network graphs with centrality metrics per match window
- Dynamic role assignment output per match frame
- Tactical phase labels with CRF/Viterbi constraint
- Semantic attribute JSON exported per match for Project 3
- FastAPI with documented endpoints for Projects 2 and 3
- MLflow experiment logs for all ablation studies
- PostgreSQL schema with all tables populated
- Docker Compose setup for full local run
- Google Cloud Run deployment with CI/CD

PROJECT REQUIREMENT DOCUMENT

Project 2 — Dynamic Passing & Formation Visualizer

Visual Analytics Engine | Frontend Dashboard

1. Project Identity

Field	Detail
Sub-project title	Dynamic Passing & Formation Visualizer — Interactive Tactical Dashboard
Project type	Frontend-heavy visual analytics system
Role on resume	Frontend Developer / Data Visualizer / Analytics Engineer
Depends on	Project 1 FastAPI backend for all computed data
Priority	Second — build after Project 1 API is stable

2. What This Project Is

A fully interactive, animated football tactical board — not a notebook, not a matplotlib plot, not a static dashboard. A web application that feels like a professional football analysis tool. Think FC26 tactical view meets data analytics platform.

It consumes all computed outputs from Project 1 and presents them through animated passing networks, moving Voronoi cells, formation overlays that morph between detected tactical shapes, player cards with metrics, a match timeline scrubber, convex hull animations, tables, and charts.

3. Core Features

Tactical Pitch Board

- Interactive SVG/Canvas football pitch rendered in the browser
- Players shown as positioned nodes that move between tactical phases
- Formation shape overlaid as a connected polygon that morphs smoothly
- Pressing intensity shown as color gradients on pitch zones
- Defensive block and compactness boundary rendered as animated convex hull

Passing Network View

- Directed weighted graph rendered on the pitch
- Edge thickness = pass frequency
- Edge color = xT contribution of that passing lane
- Node size = centrality score of that player
- Temporal decay toggle — see which connections were active in which phase
- Animation showing network evolution across match phases

Formation and Role View

- Detected formation label per phase
- Role assignment overlay — each player labeled with their current tactical role
- Role-swap events highlighted with animated transition
- Side-by-side comparison of two teams' formations

Spatial Control View

- Voronoi cells rendered per frame — each player's territorial zone
- Animated as match phase changes
- Color coded by team
- Off-ball run zones highlighted

Player Analytics Panel

- Individual player card with xT, VAEP, centrality metrics
- Heatmap of touch and action locations on pitch
- Pass map for selected player
- Comparison slider between two players

Match Timeline

- Horizontal scrubber showing full match timeline
- Phase labels shown across the bar
- Click any phase to jump the tactical board to that state
- Goal, substitution, card events marked on timeline

Stats Tables and Charts

- Sortable table for all players with all computed metrics
 - Bar and radar charts for team comparison
 - Line chart showing xT accumulation over time
 - Passing network density over match phases
-

4. System Architecture

Frontend — React + TypeScript

- Pitch Board Component → D3.js + SVG animated pitch
- Passing Network Component → Cytoscape.js directed graph
- Voronoi/Spatial Component → Deck.gl GPU-accelerated overlay
- Formation Overlay → D3.js animated polygon morphing
- Player Panel Component → Recharts + custom cards
- Timeline Scrubber → Framer Motion + Zustand state
- Stats Table Component → TanStack Table
- Match Selector → dropdown + search UI

State Management

- Zustand store → selected match, phase, player, team

API Communication

- React Query → cached data fetching from Project 1 API
- Axios → HTTP client for FastAPI endpoints

Build and Deployment

- Vite → dev server + production build
 - Tailwind CSS → styling
 - Vercel → deployment with CDN
 - GitHub Actions → CI/CD on push
-

5. Pages and Views

Page	Content
Home / Match Selector	Search and select a match from the database
Tactical Board	Main animated pitch view with all overlays

Passing Networks	Full-screen network graph view
Formation Analysis	Formation morphing and role assignment view
Player Analytics	Individual player deep-dive panel
Team Comparison	Side-by-side two-team tactical comparison
Match Report	Static printable summary view

6. API Endpoints Consumed from Project 1

Endpoint	Data returned	Used by
GET /matches	List of all available matches	Match selector
GET /match/{id}/phases	Tactical phase labels + timestamps	Timeline scrubber
GET /match/{id}/passing-network	Nodes, edges, weights per phase	Passing network view
GET /match/{id}/formation	Formation label + player positions per phase	Formation overlay
GET /match/{id}/roles	Role assignments per player per frame	Role view
GET /match/{id}/spatial	Voronoi, convex hull, centroid data	Spatial view
GET /match/{id}/players	xT, VAEP, centrality per player	Player panel + tables
GET /match/{id}/timeline	Full event timeline with metadata	Timeline component

7. Tech Stack

Layer	Choice
Framework	React 18 + TypeScript
Pitch rendering	D3.js + SVG

GPU overlays	Deck.gl
Graph visualization	Cytoscape.js
Charts	Recharts + Nivo
Tables	TanStack Table
Animation	Framer Motion
Canvas tactical board	Konva.js
State management	Zustand
Data fetching	React Query + Axios
Styling	Tailwind CSS
Build	Vite
Deployment	Vercel
CI/CD	GitHub Actions

8. Diagrams to Create

1. UI component hierarchy diagram
 2. Data flow diagram — API response to state to rendered component
 3. Page and view map — navigation structure
 4. Wireframe sketches — tactical board layout, player panel, timeline
-

9. Deliverables

- Fully deployed React web application on Vercel
- Animated tactical pitch board with formation morphing
- Interactive passing network with temporal decay toggle
- Voronoi spatial control overlay
- Player analytics panel with xT/VAEP/centrality
- Match timeline scrubber with phase